torchsimus: un simulador diferenciable de ultrasonido médico en PyTorch

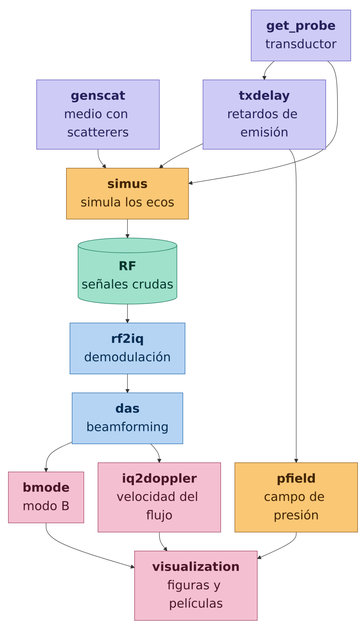
Pelao Oneto, Ronal Coronado
Facultad de Ingeniería y Negocios, NIDS

1. Introducción

La simulación de ultrasonido médico es una herramienta clave para diseñar transductores, probar estrategias de emisión y generar datos para entrenar algoritmos de diagnóstico por imagen sin necesidad de pacientes. SIMUS, parte de la toolbox MUST [1, 2], es un simulador de referencia validado [3]; sin embargo, está escrito en MATLAB, y su versión en Python (pymust) usa NumPy, por lo que ninguna de las dos puede usar GPU ni calcular derivadas. En este trabajo presentamos torchsimus, una reimplementación de SIMUS en PyTorch [4] que ejecuta la misma física en CPU o GPU, procesa lotes de simulaciones y es completamente **diferenciable**: permite calcular gradientes respecto a la reflectividad, la posición de los scatterers o los retardos de emisión. torchsimus es un paquete de Python de código abierto, orientado a la investigación en imagenología médica, que se instala con `pip install git+https://github.com/GianfrancoOneto/torchsimus.git`.

2. Flujo de simulación

Una simulación reproduce los pasos de un ecógrafo real. Primero se definen el transductor (`get_probe`), el medio con sus scatterers (`genscat`) y los retardos de emisión (`txdelay`); luego `simus` calcula los ecos que recibe cada elemento (señales RF), y la imagen se forma mediante demodulación (`rf2iq`) y beamforming (`das`).



3. Modelo físico

El modelo trabaja en el dominio de la frecuencia. Para un scatterer s ubicado a una distancia r_{sn} del elemento n, la función de Green es

$$G_{sn}(f) = \frac{e^{ikr_{sn}}}{\sqrt{r_{sn}}}\, D_n(\theta, f), \qquad k = \frac{2\pi f}{c},$$

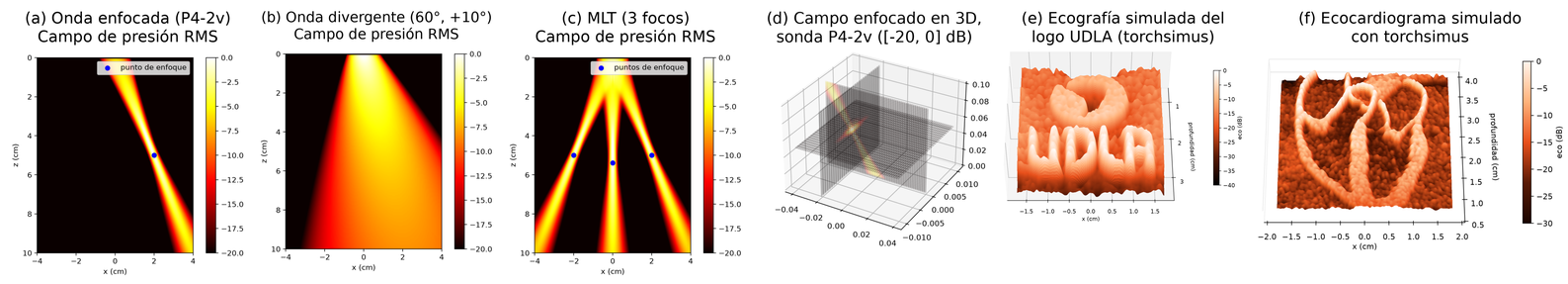
donde D_n es la directividad del elemento y θ el ángulo respecto de su normal. Si cada elemento emisor m tiene apodización w_m y retardo τ_m, el espectro **pulso-eco** que recibe el elemento n es

$$Y_n(f) = Q(f) \sum_s G_{sn}\, R_s \sum_m G_{sm}\, w_m e^{i2\pi f \tau_m},$$

donde R_s es la reflectividad y $Q(f)$ el espectro pulso-eco de la sonda. Las señales RF se obtienen con una FFT inversa. Como cada paso es una operación con tensores de PyTorch, los gradientes se calculan automáticamente.

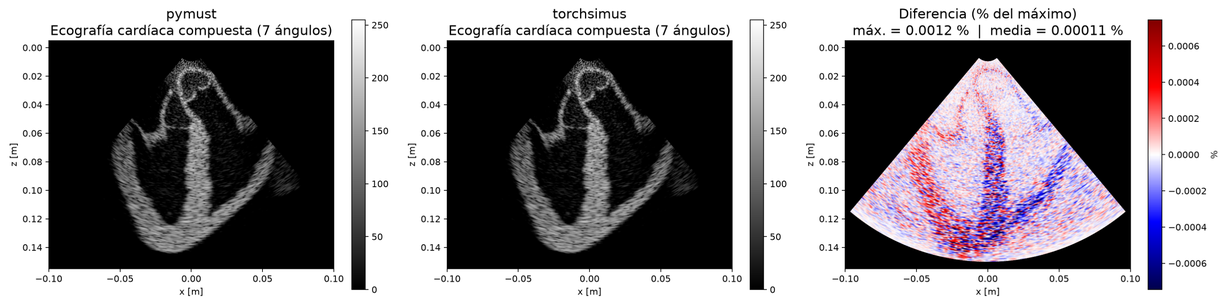
4. Aplicaciones

torchsimus es una herramienta versátil, con múltiples aplicaciones en imagenología médica. Calcula campos de presión y señales RF para arreglos lineales, convexos, matriciales, en anillo y personalizados, con emisiones enfocadas, planas, divergentes y de múltiples focos (MLT). La cadena completa de formación de imagen también está disponible en PyTorch: demodulación (`rf2iq`), beamforming delay-and-sum (`das`), modo B, Doppler color y *speckle tracking*. Abajo se muestran campos de presión RMS (dB) en 2-D (a–c) y en 3-D (d), una ecografía realizada sobre el logo de la UDLA (e) y un ecocardiograma (f), todos simulados con torchsimus.



5. torchsimus frente a pymust

torchsimus entrega **los mismos resultados que pymust** y suma lo que pymust no tiene: GPU, gradientes y lotes. Abajo se compara la misma ecografía cardíaca compuesta (sonda P4-2v, 7 ondas divergentes, ∼28 000 scatterers) simulada con ambos: las imágenes son indistinguibles, con una diferencia máxima de 0.0012 % respecto de la intensidad máxima.



	pymust	**torchsimus**
Física de SIMUS	✓	✓
Cálculo en GPU	×	✓
Diferenciable (gradientes)	×	✓
Lotes de simulaciones	×	✓
Uso con redes neuronales	×	✓

Tiempo	pymust	**torchsimus**	
	CPU	CPU	GPU
Ecografía cardíaca (RF)	395 s	257 s	**17.2 s**
Película `mkmovie`	75 s	41 s	**0.7 s**
Campo 3-D, matriz 32×32	–	349 s	**7.2 s**
Onda plana con atenuación	–	105 s	**2.5 s**
Subaperturas L11-5v	–	64 s	**1.9 s**

CPU: 1 hilo. GPU: NVIDIA Tesla T4. –: no medido.

Comparación con pymust	Diferencia
Campo de presión (12 escenas)	$\leq 9 \times 10^{-6}$
Campo 3-D, matriz 32×32	9×10^{-7}
Señales RF, corazón	4×10^{-5}
Modo B compuesto	0.0004 dB
Doppler, datos reales	0.000 cm/s

Error relativo, salvo que se indique la unidad.

El paquete aprueba **96 pruebas automáticas**.

6. Conclusiones

torchsimus reproduce los resultados de pymust (diferencias de $\sim 10^{-5}$) y amplía sus capacidades:

1. Calcula campos de presión, señales RF e imágenes (modo B, Doppler) para sondas 2-D y 3-D.
2. Es diferenciable: se pueden optimizar con gradientes los retardos de emisión, la posición de los scatterers o la reflectividad.
3. Corre en GPU y en lotes, lo que permite generar rápidamente datos para aprendizaje profundo en imagenología médica.

Código en GitHub

7. Referencias

[1] Damien Garcia. SIMUS: an open-source simulator for medical ultrasound imaging. Part I: theory & examples. *Computer Methods and Programs in Biomedicine*, 218:106726, 2022.

[2] Damien Garcia. MUST: Matlab UltraSound Toolbox. `https://www.biomecardio.com/MUST`.

[3] Amanda Cigier, François Varray, and Damien Garcia. SIMUS: an open-source simulator for medical ultrasound imaging. Part II: comparison with four simulators. *Computer Methods and Programs in Biomedicine*, 220:106774, 2022.

[4] Adam Paszke et al. PyTorch: an imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.